

Effective and Efficient Big OLAP Data Cube Compression in Mobile Cloud Environments: The IQTS Algorithm

Alfredo Cuzzocrea^{1,*}
iDEA Lab

University of Calabria
Rende, Italy
& Dept. of Computer Science
University of Paris City
Paris, France
alfredo.cuzzocrea@unical.it

Mojtaba Hajian
iDEA Lab
University of Calabria,
Rende, Italy
mojtaba.hajian@unical.it

Abstract—This paper introduces the *Indexed Quad-Tree Summary (IQTS)* algorithm, along with its concepts, models, and “philosophy”. IQTS allows us to compress *multidimensional OLAP views* derived from *big OLAP data cubes* that populate *Mobile Clouds*. In addition to this, IQTS effectively and efficiently supports *approximate query answering over compressed multidimensional OLAP views*. These functionalities turn to be *enabling functionalities* for big data applications in a wide collection of modern scenarios, such as *healthcare analytics*. In light of these considerations, this paper provides motivations, anatomy and procedures of IQTS, enriched by several case studies that contribute to pinpoint the benefits coming from our proposed algorithm.

Keywords—*Big Data, Big OLAP Cubes, Mobile Clouds, Approximate Query Answering, Intelligent Big Data Applications*

I. INTRODUCTION

Mobile Cloud Environments (e.g., [1,2,3]) are conquering the scene, even because coupled with *big data research initiatives* (e.g., [4,5,6]). Mobile Clouds are now very popular, as they characterize a plethora of modern intelligent applications, ranging from environmental monitoring, healthcare support, online auction, hazardous/extreme environments, pharmacovigilance, and so forth.

Indeed, *ubiquitous computing* (e.g., [7,8]) is now everywhere, especially with the tremendous rising of ICT technologies, based on which we now use an impressive number of *mobile devices* including smartphones, tables, personal digital assistants, and so forth. At the same, the volume, velocity and variety of big data is exploding day by day, therefore accessing these datasets to retrieve information and knowledge via mobile devices is a critical open issue in modern big data research (e.g., [9]).

On a wider perspective, these systems can be intended as *mobile big data systems*, since they really rely on big datasets, with the goal of finally supporting actionable knowledge discovery and intelligent decision making (e.g., [10]). Due to this evident property, *managing, querying and analyzing big datasets in these systems* has become a real research challenges, especially due to nature of big data, complexity of big data, topology of the network/delivery model, data availability, and so forth (e.g., [11]).

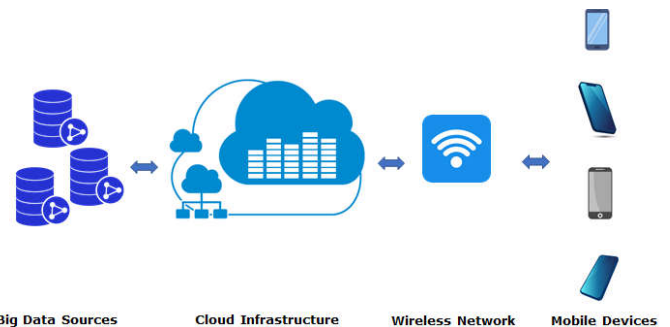


Fig. 1. Mobile big data Cloud environment.

Fig. 1 shows a typical *mobile big data Cloud environment*, where massive *Big Data Sources* are made available via a proper *Cloud Infrastructure*. In turn, the Cloud is accessed via a *Wireless Network* by several *Mobile Devices*, which finally access, manage, query and analyze target datasets.

Among the various family of big data sources, *big OLAP data cubes* play an astonishing role (e.g., [12]). Indeed, these *multidimensional structures* store (big) data at a *high number of dimensions*, as required by modern big data application scenarios, and provide support to a plethora of powerful analytics that can be developed on top of them (e.g., [13,14,15]). On the other hand, their size and complexity make prohibitive the cost for accessing and querying them, so that *data compression paradigms* are necessary in order to mitigate this relevant drawback. Also, the presence of mobile devices makes the problem worse, due to their well-known limitations (e.g., [16]).

Inspired by these considerations, this paper introduces the *Indexed Quad-Tree Summary (IQTS)* algorithm, along with its concepts, models, and “philosophy”. IQTS allows us to compress *multidimensional OLAP views* derived from *big OLAP data cubes* that populate *Mobile Clouds*. In addition to this, IQTS effectively and efficiently supports *approximate query answering over compressed multidimensional OLAP views*. These functionalities turn to be *enabling functionalities* for big data applications in a wide collection of modern scenarios, such as *healthcare analytics*, including approaches inspired from visualization metaphors (e.g., [17]). We provide motivations, anatomy and procedures of IQTS, enriched by several case studies that contribute to pinpoint the benefits coming from our proposed algorithm.

* This research has been done in the context of the Excellence Chair in Big Data Management and Analytics at University of Paris City, Paris, France

II. THE IQTS ALGORITHM

In this Section, we provide a detailed description of the IQTS algorithm.

As noticed in Section I, computational cost is a drawback in multidimensional OLAP data cubes, however, there is an approach to use handheld devices as a solution. Today, mobile devices have been very popular and every people possess a mobile device. IQTS arises to enable mobile users (m -users) to access, query and receive approximate answers from remote OLAP servers, using intelligent compression techniques. IQTS applies a partitioned representation of the OLAP view to create a compressed two-dimensional OLAP view which can be downloaded into the handheld device, then the m -user is able to make a query off-line.

Fig. 2 illustrates the procedure in which a handheld device uses the data cubes in an OLAP server. For this goal, the steps are as follows:

- considering a multidimensional data cube C , the m -user selects two dimensions of the data cube *Visualization Dimensions* (VD). Each VD is made from merging of other dimensions of the data cube C ;
- since we have a new data cube, we need to obtain new OLAP aggregations, then a two-dimensional OLAP view of C is created which is designated V ;
- a compressed representation of V is created in an application server, then it is transmitted to the m -user.

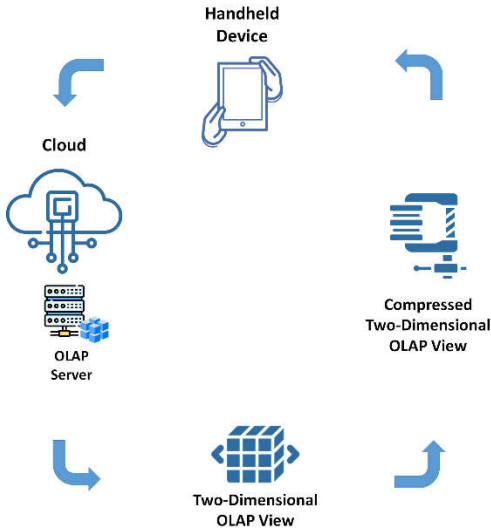


Fig. 2. Data flow procedure in the IQTS algorithm.

OLAP compression process has a number of steps including: (i) *dimension flattening*; (ii) *compression of two-dimensional OLAP view* (iii) *in-memory representation*.

The first step of the compression technique is *dimension flattening*. Consider two dimensions d_{j+1} and d_j , with the hierarchies $X(d_{j+1})$ and $X(d_j)$, to be merged together and create a new dimension hierarchy called $X^*(d_j)$, as it is shown in Fig. 3. In this case, a sub-tree of $X(d_{j+1})$ rooted at the root node of $X(d_{j+1})$ with a depth of $D_{j+1} = 1$ is selected, then a clone of it is inserted to each member of the level L_{X_j} of $X(d_j)$ at the depth $L_{X_j} = 1$. Therefore, the $X^*(d_j)$ is resulted from merging of the two dimensions d_j and d_{j+1} . We can continue this procedure to merge other dimension of the multidimensional OLAP, C , with $X^*(d_j)$, so that flatten more than two dimensions to one new dimension.

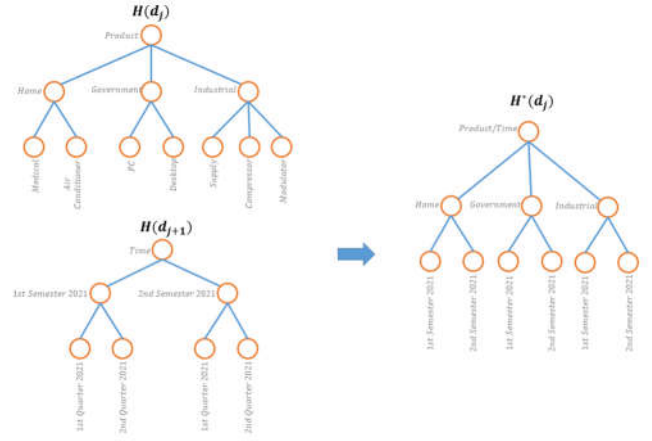


Fig. 3. An example of dimension flattening (merging two dimensions to one dimension).

After the flattening procedure, it should be noted that the data must be re-aggregated according to the new VD s.

The main compression technique applies on the resulted two-dimensional OLAP view V after *dimension flattening*. The compression procedure applies iterative methods to develop a quad-tree partition of V , where each bucket contains the sum of the items exist inside it. The algorithm targets to select the bucket p_j where it includes the least homogeneity calculated by the following criterion:

$$SSE(p) = \sum_{i \in p} (V[i] - AVG(p))^2 \quad (1)$$

where i denotes a position inside p , $V[i]$ is the value of V at position i , and $AVG(p)$ is the average of values inside p .

To split the buckets into four new buckets with less homogeneity, the algorithm of splitting finds a *splitting position* ($N_{j,k}$). Consider dimensions d_0 and d_1 of the OLAP view V , the selected bucket p_j has a range $[l_{j,0}:r_{j,0}]$ on dimension d_0 and $[l_{j,1}:r_{j,1}]$ on dimension d_1 , then $N_{j,k}$ is selected from $[l_{j,k}:r_{j,k}]$. Thus, the interval is divided to two parts: $[l_{j,k}:N_{j,k}]$ and $[N_{j,k} + 1:r_{j,k}]$, where $N_{j,k}$ is such a splitting point at a level of $X(d_k)$.

Firstly, we select the half-way point of the interval $[l_{j,k}:r_{j,k}]$ as follows:

$$H_{j,k} = \lfloor (r_{j,k} - l_{j,k} + 1)/2 \rfloor \quad (2)$$

and designate $\alpha_{j,k}$. Now, considering the $L_{j,k}$ as a specific level of the tree $T_{j,k}$, we have the $\beta_{j,k}$, the ancestor of $\alpha_{j,k}$ at the level $L_{j,k}$. Moreover, $\alpha_{j,k}^L$ and $\alpha_{j,k}^R$ are the *Leftmost Descendant Leaf* (LDL) and *Rightmost Descendant Leaf* (RDL) of $\beta_{j,k}$, respectively. We define $\alpha_{j,k}^*$ of $\{\alpha_{j,k}^L, \alpha_{j,k}^R\}$, where there are two states as following if it is inside or outside the interval:

$$\emptyset_{j,k} = [H_{j,k} - \lfloor \frac{1}{3}(r_{j,k} - l_{j,k} + 1) \rfloor : H_{j,k} + \lfloor \frac{1}{3}(r_{j,k} - l_{j,k} + 1) \rfloor] \quad (3)$$

noting that we intend to select a splitting position close to the half-way point of the $[l_{j,k}:r_{j,k}]$:

- if $\alpha_{j,k}^*$ is inside the $\emptyset_{j,k}$: then the $\alpha_{j,k}^*$ is the *splitting position* of $[l_{j,k}:r_{j,k}]$;

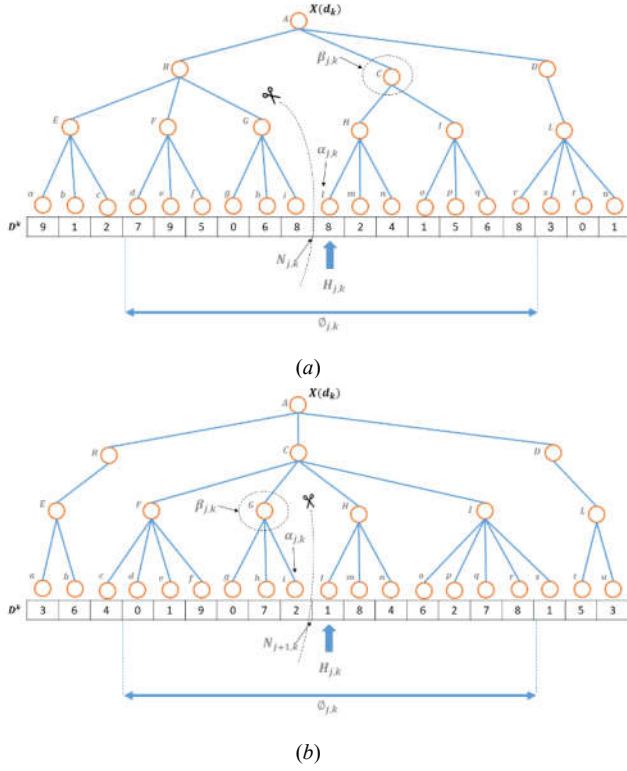


Fig. 4. Two examples of hierarchy splitting.

- if $\alpha_{j,k}^*$ is outside the $\emptyset_{j,k}$: then select another $\beta_{j,k}$ where it is an ancestor node of $\alpha_{j,k}$ at a deeper level of the hierarchy. This procedure is repeated until an appropriate $\alpha_{j,k}^*$ inside the $\emptyset_{j,k}$ is found. Then the $\alpha_{j,k}^*$ is the *splitting position* of $[l_{j,k}:r_{j,k}]$.

The above procedure is shown in Fig. 4 as two examples of splitting position $N_{j,k}$.

From Fig. 5 to Fig. 8, we show the compression procedure of the last example two-dimensional OLAP view V Zone/Time-Product dimensions. In Fig. 5, the algorithm considers one bucket where represented as the whole OLAP view V . At Fig. 6, each dimension is split to two new partitions, thus there are four new buckets. Note that the target is to select the buckets with least homogeneity (i.e. containing the least uniform distribution of data), and split them to make new homogenous buckets. In the next step, we have new buckets that should be investigated for their homogeneity, and the least homogenous buckets must be divided to four new buckets; as it is shown in Fig. 7, one bucket from last partitioning step needs to be split. In the next steps this procedure is continued with the above criteria (see Fig. 8). The procedure will be continued until the storage assigned in the handheld device is full.

The representation of compressed OLAP view is an important issue, because of the memory limitations in mobile systems. Thus, an approach is necessary to make use of the available memory in an efficient way. In this approach, we assume that only non-negative integers are included in the OLAP view V , and each bucket contains the sum of the measure values inside it, where it is assigned 32 bits for these values. After splitting procedure, each bucket is split to four sub-buckets, then we store the sum of the values existing inside each three of the new sub-buckets (if the sum value is non-zero; if the sum value is zero then we leave 32 bits as null

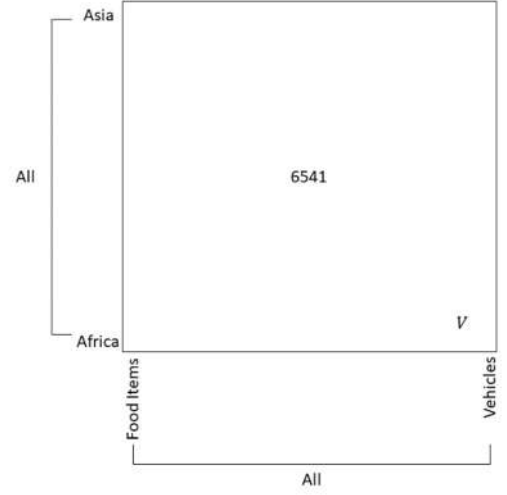


Fig. 5. Partitioning procedure of an example two-dimensional data cube: step 1/4.

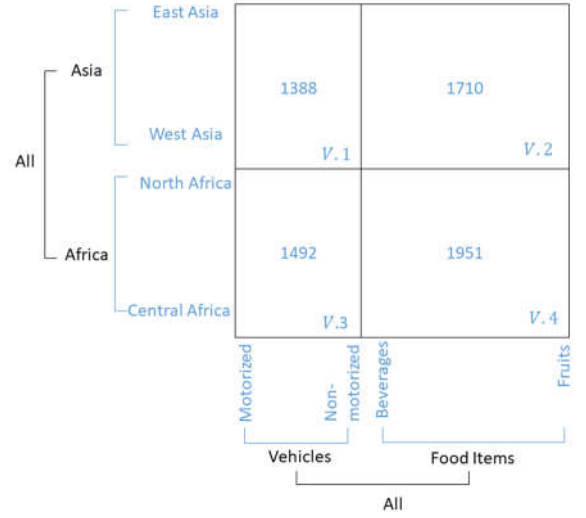


Fig. 6. Partitioning procedure of an example two-dimensional data cube: step 2/4.

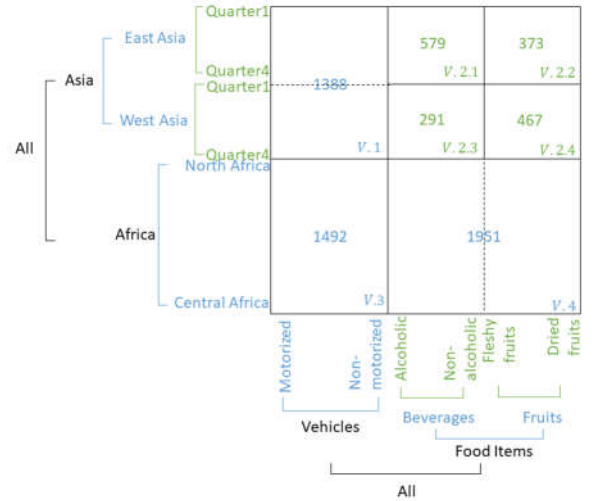


Fig. 7. Partitioning procedure of an example two-dimensional data cube: step 3/4.

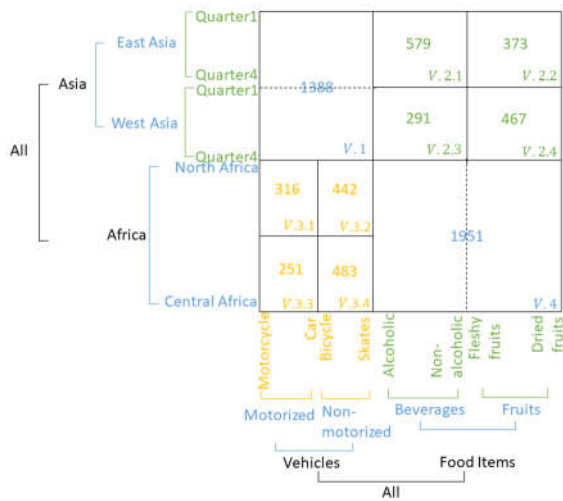


Fig. 8. Partitioning procedure of an example two-dimensional data cube: step 4/4.

in order to use that for other additional buckets) and leave the fourth one. Using this approach, the fourth bucket's value can be calculated by subtraction of sum of the three sub-buckets from the parent bucket's value. In the next steps, the splitting procedure is continued as selecting the least uniform distributed data to be split to four buckets. Therefore, the elements inside each new bucket will have small variance and the values will be close to the average of the elements' values inside the bucket.

Our technique, *at this step* (i.e., compression *after* the first-stage splitting described above), is inspired by [18] and produces a *quad-tree partition* of V , finally represented by IQTS, which singles-out a number of buckets and stores the sums of measure values only for them. More into details, we are given a two-dimensional data cube view V of size N , already partitioned in four buckets. We want to construct a compressed representation of V using a size $M \ll N$. Such a partitioning scheme is based on choosing, at each step, as that to further partitioning the bucket having the maximum *Sum of Square Errors* (SSE) between the original data cube view V and its *current* partitioned representation (i.e., $IQTS(V)$), defined as follows:

$$SSE(IQTS(V)) = \sum_{q_i \in Leaves(IQTS(V))} SSE(q_i) \quad (4)$$

such that $Leaves(IQTS(V))$ is the set composed by the leave nodes of $IQTS(V)$, also called *terminal buckets*, and:

$$SSE(q_i) = \sum_{j \in q_i} (V[j] - AVG(q_i))^2 \quad (5)$$

In addition to the above issue, there is another challenge that should be considered. In the query estimation, the reconstruction of the data after compression process, can be performed by a linear interpolation if the distribution of data in each bucket is uniform. But, the problem is that practically this will not happen, it means that the buckets don't contain uniformly distributed enough to enable us to use linear interpolation for query estimates. To solve this challenge, we need to store some information that enhances the approximation accuracy in data distribution reconstruction of each buckets. The *Indexing* method is used to address the problem of data distribution. For the sake of efficiency, *Indexing* method only is applied on the non-zero sum buckets and/or buckets containing data with not very uniform

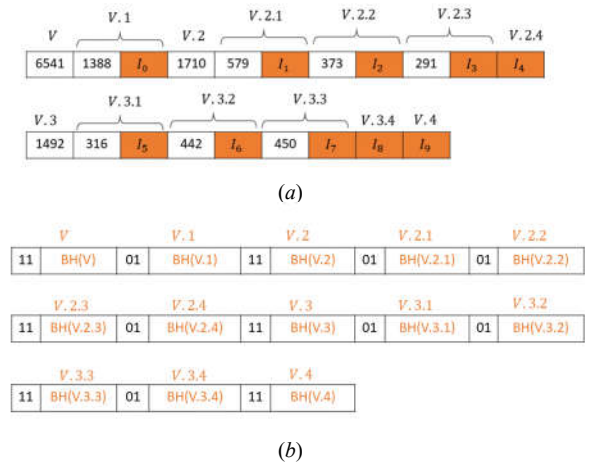


Fig. 9. Sum Array (a) and Structure Array (b) of the IQTS.

distribution, because this type of data distribution does not need linear interpolation and does not enhance the accuracy.

There are three Indexing methods proposed to select the best approach to that approximates the bucket's data distribution: (i) the *2/3LT-Index*, is appropriate if the distributions don't contain strong asymmetry; (ii) the *2/4LT-Index*, is appropriate if the distributions are biased; (iii) the *2/p(eak)LT-Index*, is appropriate if the distributions contain a few high-density peaks. The purpose of the *Index* is approximating the aggregate (sum) of the data exists in the sub-buckets and sub-sub-buckets. Finally, after performing above steps the *Sum Array* and *Structure Array* are prepared to be stored in memory as shown in Fig. 9 for the aforementioned OLAP view example.

Finally, the information about the values of the nodes and the structure of partitioned quad-trees is stored in the memory as shown in Fig. 9. The *Sum Array* contains the sum value of each node if it is non-zero besides the *Index* that is an approximation of the data inside sub-buckets.

The *Structure Array* represents the partitioning process of the two-dimensional OLAP view V , for the node n_j corresponding to bucket p_j , using (i) two bits assigned for *Topology Header*, $TH(p_j)$, that the first bit denotes whether node n_j is a leaf or not; and the second bit represents whether the sum value is zero or not; (ii) *Bound Header*, is a number of bits designated by $BH(p_j)$ to represent the partitioning procedure of each buckets. The bounds of the child buckets will be represented by the splitting position $N_{j,k}$ on the interval $[l_{j,k}, r_{j,k}]$ of p_j , using a signed offset $O_{j,k}$ which the sign is respect to the $H_{j,k}$.

As mentioned before, IQTS also provides support for approximate query answering over the compressed OLAP views.

Range queries are evaluated on compressed OLAP views by performing *linear interpolation*, i.e. by assuming that data distribution inside each bucket is uniform (i.e., the *Continuous Value Assumption* (CVA) [19]); as a consequence, the contribution of the buckets that partially overlap the ranges of the input query is generally approximate, unless the original distribution of frequencies inside these buckets is actually uniform.

Our query algorithm navigates the quad-tree searching for: (i) all *maximal* buckets which are completely involved in the query, and (ii) all buckets which are partially included in the range of the query, and which are not split (such buckets

correspond to leaves of the quad-tree (i.e., the terminal buckets). The former buckets do not introduce any approximation, since for each of them the exact value of the sum inside the corresponding ranges is stored. On the contrary, the latter ones introduce approximation, since we cannot re-construct the exact distribution of the original data contained in the corresponding ranges. It should be noted here that, dealing with approximation in query answering, is a significant research challenge (e.g., [20]).

Linear interpolation is used to answer overlapping queries against a bucket b in both the cases that b is equipped with an *Index* or not. In more detail, given a bucket b and a range query Q overlapping b (i.e., $|Q \cap b| \neq \emptyset$), if b is not equipped with an *Index*, we compute the approximate answer to Q on b $\tilde{A}(Q)$ by applying the CVA:

$$\tilde{A}(Q) = \frac{|Q \cap b|}{|b|} \cdot S(b) \quad (6)$$

such that $S(b)$ is the sum stored in b , $|Q \cap b|$ is the area on the intersection region between Q and b , and $|b|$ is the area of b .

III. A REFERENCE CASE STUDY: *ADVENTURE WORKS CYCLES*

In this Section, we describe *AdventureWorks*, which is a synthetic multidimensional database storing data produced by a fictitious large multi-national manufacturing company called, not by chance, *Adventure Works Cycles*. From this latter one, we then extract a multidimensional view in order to achieve the final data cube of interest. This data cube is used as case study for providing a complete case study on how IQTS works in practice (presented in Section IV). The remaining part of this Section is organized as follows. First, we provide a short background on *AdventureWorks*, by describing the various application scenarios. We investigate several sectors, particularly those related to sales and marketing, shopping and vendors, products, and finally we focus on the production. We also investigate the database requirements, along which detailed information on represented data and relevant areas. We later provide the description on schemas of main correlated tables. Finally, we describe the data cube of interest, by showing the related multidimensional view from which it is obtained, along which the target dimensions and hierarchies.

The synthetic *multidimensional database* *AdventureWorks* refers to the fictitious large multi-national manufacturing company *Adventure Works Cycles*, which manufactures and sells metal and composite bicycles to North American, European and Asian commercial markets. While its base operation is located in Bothell, Washington with 290 employees, several regional sales teams are located throughout their market base.

In 2000, *Adventure Works Cycles* bought a small manufacturing plant, which is *Importadores Neptuno*, located in Mexico. This latter manufactures several critical mounting components for the *Adventure Works Cycles* product line, which are shipped to the Bothell office for final product assembly. In 2001, *Importadores Neptuno*, became the sole manufacturer and distributor of the touring bicycle product group.

Coming off a particularly successful year, *Adventure Works Cycles* is looking to broaden its market share by targeting sales to their best customers, by extending their product availability through an external Web site, and reducing their cost of sales through lower production costs.

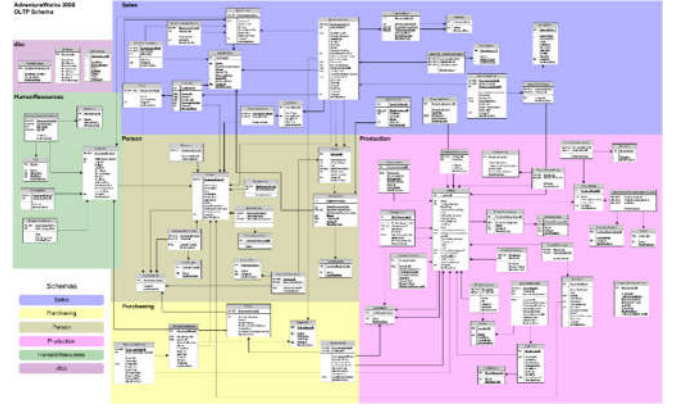


Fig. 10. *AdventureWorks* multidimensional database schema.

Adventure Works Cycles targets the following types of customers:

- individuals, which are consumers who buy products from the *Adventure Works Cycles* online store;
- stores, which are retail or wholesale stores that buy products for resale from *Adventure Works Cycles* sale representatives.

The product lines can be classified according to the following four categories:

- bicycles manufactured by *Adventure Works Cycles*;
- bicycle components, which are spare parts such as pedals, wheels and brakes;
- cycling wear purchased from the suppliers in order to resale to their customers;
- cycling accessories purchased from the suppliers in order to resale to their customers.

The purchasing department buys raw materials and parts used in the manufacture of *Adventure Works Cycles* bicycles. *Adventure Works Cycles* also purchases products for resale, such as bicycle apparel and bicycle add-ons like water bottles and pumps. The information about these products and vendors from whom they are obtained is stored into the multidimensional database *AdventureWorks*.

In *AdventureWorks*, tables provide support to the following typical manufacturing areas:

- manufacturing processes:
 1. bill of materials, which is a list of products used or contained in other products;
 2. work orders that are manufacturing orders by work center;
 3. locations, which defines the main manufacturing and inventory areas, such as frame forming, paint, component assembly, and so forth;
 4. manufacturing and product assembly instructions by work center.
- product inventory, which is the physical location of a product in the warehouse or manufacturing area, and the related availability;
- engineering documentation that is technical requirements and maintenance documentation for bicycles and related components.

Fig. 10 shows the schema of the *AdventureWorks* multidimensional database. Here, tables are organized into six conceptual groups: (i) *Sales*, which groups concepts that are related to sales of the company; (ii) *Person*, which

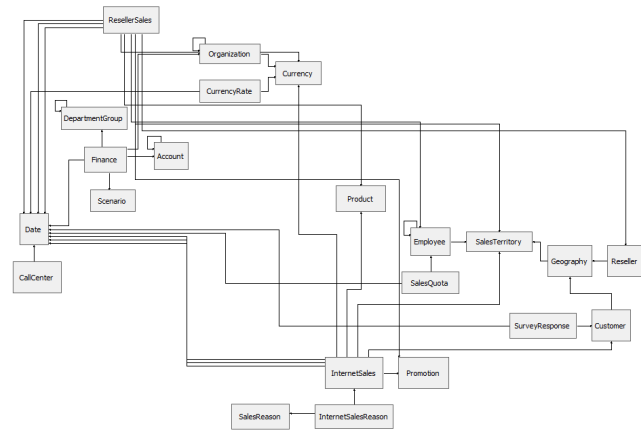


Fig. 11. The SaleAnalyzerView view from which the SaleAnalyzer data cube is derived.

collects anagraphical and logistic information on customers and operators of the company; (iii) *Production*, which groups concepts in the area of products of the company and their production/supply-chain; (iv) *Purchasing*, which collects purchase information and their details (e.g., shipping, vendors, etc.); (v) *Human Resources*, which groups concepts concerning employees, departments, and so forth; (vi) *dbo*, which collects internal database system information.

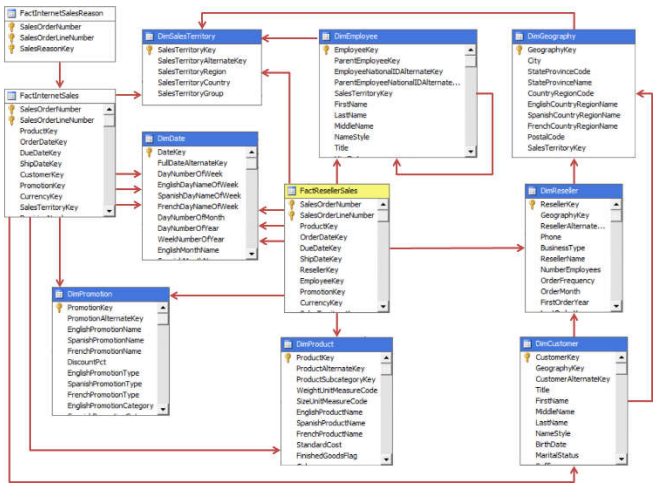


Fig. 12. SaleAnalyzer data cube snowflake schema.

The SaleAnalyzerView view shown in Fig. 11 is designed with the goal of supporting multidimensional analytics over sales data kept in the AdventureWorksDW data warehouse. On top of such multidimensional view, the SaleAnalyzer data cube (see Fig. 12) is built. In particular, SaleAnalyzer stores the following information:

- sales to retailers;
- data and related details of Internet sale orders to customers;
- shipped orders;
- data in USD, but keeping track about original currency.

SaleAnalyzer is characterized by the fact table FactResellerSales that contains 60,855 facts related to sales performed by retailers. Other tables with interesting data that are related to FactResellerSales are the following: (i) FactInternetSales, which contains 60,398 facts related to InternetSales; (ii) FactInternetSalesReason that stores 64,515 facts related to the buying motivations.

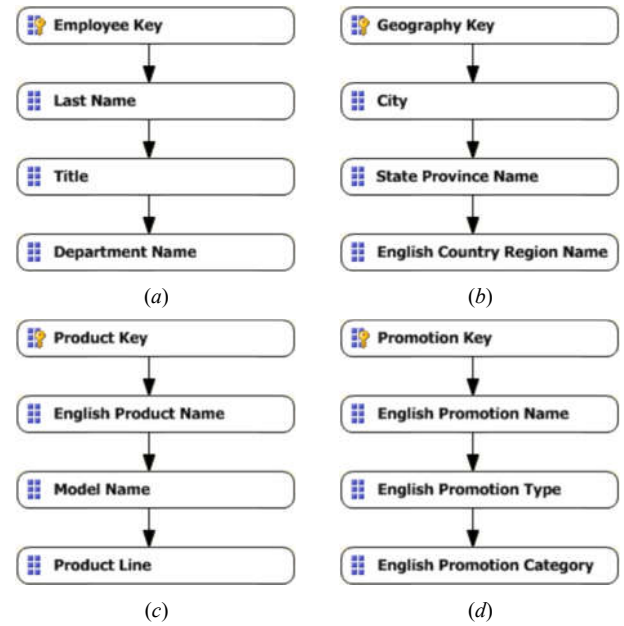


Fig. 13. Some hierarchies defined on SaleAnalyzer: (a) Employee, (b) Geography, (c) Product, (d) Promotion.

Hierarchies are later defined on each dimension. The most relevant ones are: (i) Employee in Fig. 13 (a), (ii) Geography in Fig. 13 (b), (iii) Product in Fig. 13 (c), and (iv) Promotion in Fig. 13 (d).

According to the defined hierarchies, in SaleAnalyzer, employees can be modelled according to several perspectives (dimensions, respectively), like, for instance: (i) the department they belong to (e.g., Finance, Sales, etc.); (ii) their qualification (EuropeanSalesManager, SalesRepresentative, etc.). Similarly, customers/employees can be modelled according to, for instance: (i) sales territory or birthplace within states (e.g., Australia, Canada, etc.); (ii) regions (e.g., New South Wales, Ontario, etc.); (iii) cities (e.g., Sidney, Ottawa, etc.). Again, products can be modelled by means of, for example: (i) product line (e.g., Mountain; Road, etc.); (ii) model (e.g., Full Fingers Glove; HL Mountain Frame, etc.); (iii) exact product name (Full Fingers Glove L; HL Mountain Frame - Black, 38, etc.). Finally, regarding to promotions, some possible modelling perspectives are: (i) category (e.g., No Discount; Reseller, etc.); (ii) type (e.g., Volume Discount; Discontinued Product, etc.); (iii) name (e.g., Sport Helmet Discount-2003; Touring-3000 Promotion, etc.).

IV. IQTS AT WORK!

In this Section, we provide an example of IQTS on a two-dimensional view obtained from SaleAnalyzer presented in Section III. We show the query for producing the view, thus, we describe the splitting procedure and illustrate it in the form of tree. Also, we provide the amount of memory storage used by the whole view, in more detail, we display the bits for each bucket. Finally, we provide examples on querying compressed OLAP views derived from SaleAnalyzer.

We obtained the two-dimensional view on SaleAnalyzer starting from the query reported in Fig. 14, by selecting as dimensions Geography and Product, and drilling down to the lower level in the hierarchies (as shown in Fig. 13 (b) and Fig. 13 (c)). On Geography, we chosen the cities, alphabetically, in the range from Bonn to York and, on Product, we selected the English product names, alphabetically, in the range from


```

SELECT
{[DimGeography].[Hierarchy].[City].&[Bonn]}:
[DimGeography].[Hierarchy].[City].&[York]}
ON AXIS (0)
{[DimProduct].[Hierarchy].[English Product
Name].&[Cable Lock]:
[DimProduct].[Hierarchy].[English Product
Name].&[Touring Pedal]}
ON AXIS (1),
FROM SaleAnalyzer
WHERE [Measures].[Total Product Cost]

```

Fig. 14. Query on SaleAnalyzer from which the two-dimensional view is obtained.

Cable Lock to Touring Pedal. We chosen Total Product Cost as measure.

The query result shown in Fig. 15 represents the two-dimensional view. Note that each cell stores the sum of the product costs for the delimiting products and cities range. For better understanding, we replaced the real values returned by the query with synthetic ones, specifically computed (e.g., the first cell stores the total cost of product names in the range from Cable Lock to Classic Vest L where cities are in the range of Bonn to Clay, which is 64 dollars). We also decided to limit the drawing of the algorithm iteration at a certain sum level, to avoid crowd on the figures. This means that a generic two-dimensional array cell, depicted in Fig. 15, is not found at leaf-level, thus, the compression process could proceed inside it.

As shown in Fig. 15, the different colors denote the buckets splitting performed by our algorithm. It can be seen that red parts contain more homogeneous values than the remaining parts. Thus, green and purple buckets are further split. Between the two red buckets, the four green sub-buckets and the four purple ones, the highest variance is in the orange sub- bucket, which is split even more. Finally, the last splitting is performed in the blue sub- bucket.

Cable Lock	64	132	217	93	74	98	86	110	
Classic Vest L	218	375	144	152	170	194	182	206	
Classic Vest S	93	146	219	87	98	122	110	134	
HL Crankset	129	85	112	135	194	218	206	230	
HL Headset	63	70	67	18	236	153	179	128	
LL Fork	67	69	57	63	142	249	142	216	
Minipump	68	70	80	76	89	157	203	194	
ML Headset	72	19	83	82	263	394	121	167	
Touring Pedal									
	Bonn	Clay	Essen	Hof	Kent	Mesa	Novi	Oxon	York

Fig. 15. Two-dimensional view on SaleAnalyzer originated by query shown in Fig. 14.

The last procedure is better shown in Fig. 16. In order to make it clear, we use the same colors of the two-dimensional view (shown in Fig. 15). Firstly, the view is a single bucket holding the sum 8,890. Then, the partitioning starts by splitting it into four buckets. Thus, the procedure continues on those buckets that hold the highest variance and it stops when the maximum memory threshold set for compression is reached. According to the last statement, the larger available memory storage, the higher precision in the results.

The quad-tree generated by the IQTS compression process, described above, is shown in detail in Fig. 17. We again used the same colors of the two-dimensional view, to

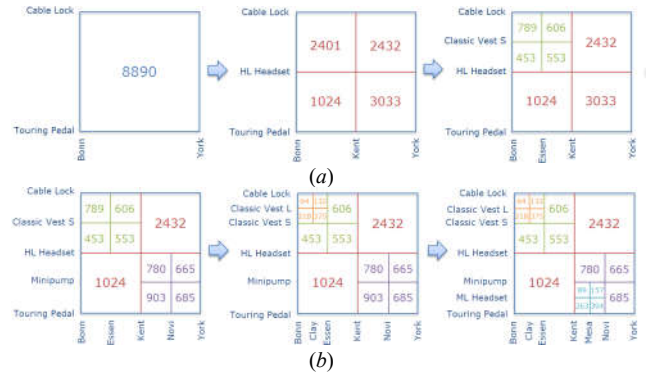


Fig. 16. Two-dimensional IQTS compression process over the two-dimensional view on SaleAnalyzer shown in Fig. 15.

get a better understanding of how the quad-tree is generated by the buckets obtained from the two-dimensional array.

In the quad-tree of Fig. 17, the nodes with non-white color represent stored buckets with non-zero sum and the nodes with white color represent not-stored buckets with non-zero sum obtained from the sums of parent and brother nodes. The number of splits executed in the compression process depends on the amount of storage space which is available in the handheld device, as will be further explained.

At the end of the described process, we obtain a summarized representation of the data cube, hierarchically organized according to a quad-tree. Each node of the quad-tree corresponds to the sum of a bucket generated during the partition, and the root is associated to the sum of the whole data cube. If the sum of the whole data cube exceeds $2^{32} - 1$, 32 bits don't suffice for storing its value. Thus, we have to

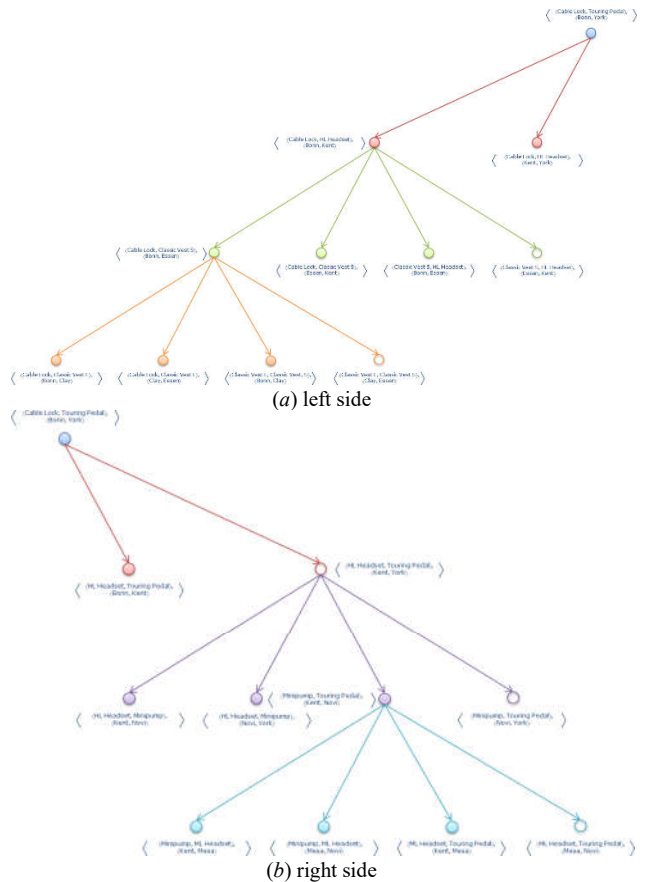


Fig. 17. Quad-tree generated by the IQTS compression process over the two-dimensional view on SaleAnalyzer shown in Fig. 15 (left side and right side).

split the data cube and, if necessary, its sub-buckets until each resulting bucket contains a sum value which can be represented using 32 bits. In such a case, the compressed representation of the data cube is a forest of quad-trees. The roots of each quad-tree are associated to distinct ranges of the raw data cube. Without loss of generality, in the following we will assume that the compressed representation of the data cube consists of a unique quad-tree.

The storage space occupied by the compressed representation of the data cube organized as a unique quad-tree consists of: (i) the space used for storing the exact values of the non-zero sums, (ii) the space used for storing the structure of the quad-tree partition.

Fig. 18 depicts the SaleAnalyzer data view related quad-tree IQTS into the secondary memory, particularly the *Sum Array*. The IQTS algorithm minimizes the information kept in memory for each bucket. For instance, the fourth bucket value is never stored since it can be calculated as difference between the parent bucket and the sum of the remaining three child buckets.

D	D.1	D.1.1	D.1.1.1	D.1.1.2	D.1.1.3	D.1.2	D.1.3
8890	2401	789	64	132	218	606	453

D.2	D.3	D.4.1	D.4.2	D.4.3	D.4.3.1	D.4.3.2	D.4.3.3
2432	1024	782	665	903	89	157	263

Fig. 18. *Sum Array* stored during the compression process over the two-dimensional view on SaleAnalyzer shown in Fig. 15.

Finally, in order to reproduce the quad-tree structure, we need to store two bits for each bucket/node, stored by the *Structure Array*. The first one indicates the node spreading, the second one indicates whether the bucket has a zero or non-zero sum. Thus, a leaf bucket/node first bit is set to 0, a parent bucket/node first bit is set to 1. For those nodes having the first bit set to 1, there is an underlying information which does not need to be stored: they spread into four children. A bucket/node which sum is equal to zero, is not shown into the sum array. In this case, the second bit is set to 0, otherwise is set to 1. Fig. 19 depicts the SaleAnalyzer data view related quad-tree IQTS into the secondary memory, particularly the *Structure Array*.

D	D.1	D.1.1	D.1.1.1	D.1.1.2	D.1.1.3	D.1.1.4
11	11	11	01	01	01	01

D.1.2	D.1.3	D.1.4	D.2	D.3	D.4	D.4.1
01	01	01	01	01	11	01

D.4.2	D.4.3	D.4.3.1	D.4.3.2	D.4.3.3	D.4.3.4	D.4.4
01	11	01	01	01	01	01

Fig. 19. *Structure Array* stored during the compression process over the two-dimensional view on SaleAnalyzer shown in Fig. 15.

Assuming that every bucket generated by the partition process has non-zero sum, and denoting the number of splits as t , the number of sum values which must be stored is $3 \cdot t + 1$. Since we use 32 bits for representing each sum, the representation of every sum occupies $96 \cdot t + 32$ bits.

The structure of the quad-tree representing the partition of the two-dimensional view can be stored using two bits for each node of the quad-tree. The value of the first of such bits indicates whether the corresponding node is a leaf or not. The

other bit indicates whether the associated sum is zero or not (if the sum is zero, it is not represented in the array of the sums). Thus, in order to store the structure of the partition we need as many bits as twice the number of nodes of the quad-tree, that is: $2 \cdot (4 \cdot t + 1)$.

To conclude, assuming that every bucket generated by the partition process has non-zero sum, the storage space occupied by the compressed representation of the data cube is $96 \cdot t + 32 + 2 \cdot (4 \cdot t + 1) = 104 \cdot t + 34$ bits.

The number of splits which can be done during the compression process is determined by the amount of available storage space. Denoting the available storage space as S , the number of splits is given by: $\frac{S-34}{104}$. Indeed, this value represents the minimum number of splits which can be performed by the compression process. Generally, since data cubes are very sparse, a lot of buckets contain only null elements, and the saved storage space can be invested for further splits.

In the next part of this Section, we provide several example on how the approximate query processing over compressed OLAP views is supported by IQTS.

In more details, we use two different query types, which produce both favourable and unfavourable case. The favourable case consists of a query returning an exact answer, so that the compression produced by the IQTS technique decreases the memory storage used by the data cube without any effects on the final result. On the contrary, in the unfavourable case, the query explores also the details in a bucket where only the sum is known. Thus, the algorithm defines the interested bucket partition (e.g., $1/4$) and approximates it such that all the values inside the bucket are homogeneous (e.g., $sum/4$).

Let's start with the best-case scenario, which is represented by queries that return exact results. Still considering the reference two-dimensional view on SaleAnalyzer shown in Fig. 15, we generate the query in Fig. 20 in order to retrieve the total cost of manufacturing the products between Cable Lock and HL Headset in the cities between Bonn and York.

```

SELECT
  {[DimGeography].[Hierarchy].[City].&[Bonn]:
  {[DimGeography].[Hierarchy].[City].&[York]}
  ON AXIS (0)
  {[DimProduct].[Hierarchy].[English Product
  Name].&[Cable Lock]:
  {[DimProduct].[Hierarchy].[English Product
  Name].&[HL Headset]}
  ON AXIS (1),
FROM   SaleAnalyzer
WHERE  [Measures].[Total Product Cost]

```

Fig. 20. Query that returns an exact answer.

In Fig. 21, we provide the logical representation of the query of Fig. 20, which is depicted in gray. In this case, the corresponding sum (5,133) entirely concerns the upper half of the two-dimensional view, i.e. the first two sub-buckets of the first bucket, and does not generate any approximation.

The query in Fig. 22 is also created ad-hoc to test a favourable case, so it returns an exact answer. This time, the query retrieves the total cost of manufacturing the products between Cable Lock and Classic Vest S in the cities between Bonn and Hof.

In Fig. 23 we provide the logical representation of the query shown in Fig. 22, still in gray. In this case, the corresponding sum (1,150) concerns three sub-buckets of the

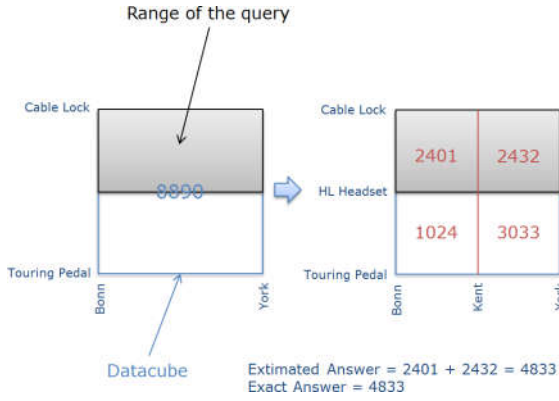


Fig. 21. Logical representation of the query in Fig. 20.

```
SELECT
{ [DimGeography].[Hierarchy].[City].&[Bonn]:
[DimGeography].[Hierarchy].[City].&[Hof] }
ON AXIS(0)
{ [DimProduct].[Hierarchy].[English Product
Name].&[Cable Lock]:
[DimProduct].[Hierarchy].[English Product
Name].&[Classic Vest S] }
ON AXIS(1),
FROM SaleAnalyzer
WHERE [Measures].[Total Product Cost]
```

Fig. 22. Query that returns an exact answer.

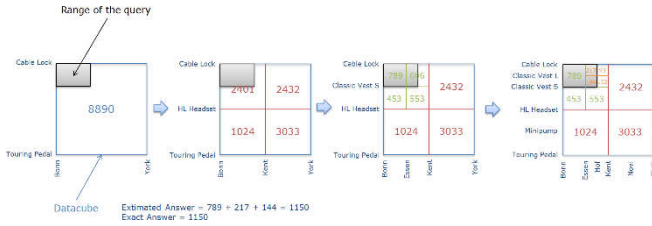


Fig. 23. Logical representation of the query in Fig. 22.

two-dimensional view and, also in this case, the result is not affected by approximation.

The answer to the query in Fig. 22 evaluated on the compressed representation of the view is exact, since the boundaries of the ranges of such queries are equal to the boundaries of some buckets generated by the compression process. That is, the ranges of the query in the left-hand side of Fig. 23 are equal to the ranges delimited by the buckets $IQTS(V).1$ and $IQTS(V).2$ (where $IQTS(V).x$ denotes the bucket x of the compressed data structure $IQTS(V)$). Analogously, the range of the query in the right-hand side of Fig. 23 is equal to the range delimited by the buckets $IQTS(V).1.1$, $IQTS(V).1.2.1$, and $IQTS(V).1.2.3$.

Contrary to the previous cases, the answer to the query in Fig. 24 is approximate, since its lower boundary does not coincide with the boundary of any bucket of the compressed representation. The final answer is the sum of the value associated to the bucket $IQTS(V).1$ with a “contribute” of the bucket $IQTS(V).3$. As reported in Section II, such a contribute is obtained by performing a linear interpolation, assuming that the elements inside $IQTS(V).3$ are uniformly distributed. This contribute, as it is clear, does not affect a typical OLAP query processing session, where the approximation is completely tolerable.

The following test concerns the unfavourable case, which is, a query that returns an approximate answer. We generate the query in Fig. 24 in order to retrieve the total cost of

```
SELECT
{ [DimGeography].[Hierarchy].[City].&[Bonn]:
[DimGeography].[Hierarchy].[City].&[Kent] }
ON AXIS(0)
{ [DimProduct].[Hierarchy].[English Product
Name].&[Cable Lock]:
[DimProduct].[Hierarchy].[English Product
Name].&[Minipump] }
ON AXIS(1),
FROM SaleAnalyzer
WHERE [Measures].[Total Product Cost]
```

Fig. 24. Query that returns an approximate answer.

manufacturing the products between Cable Lock and Minipump in the cities between Bonn and Kent.

Fig. 25 shows the logical representation of the query in Fig. 24. The part of the bucket affected by the query is shown in gray in Fig. 22. In this case, the corresponding sum (2,913) entirely concerns the first sub-bucket of the two-dimensional view and half of the third, of which only the total sum is known. In this case, the IQTS algorithm establishes the affected quantity of the third sub-bucket (e.g., $1/2$) and approximates it as if all the values in the bucket are homogeneous (e.g., $sum/4$). The fact of assuming that all the values of the bucket are homogeneous generates the approximation, since if the bucket had been investigated in depth it would have been discovered that in the upper half of the third sub-bucket the sum of the values is 474 and not 512.

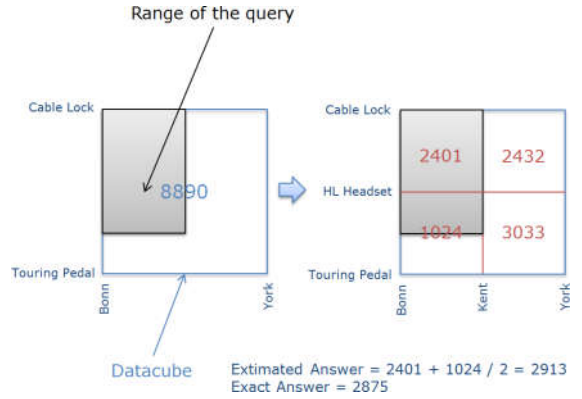


Fig. 25. Logical representation of the query in Fig. 24.

V. CONCLUSIONS AND FUTURE WORK

This paper has proposed the IQTS algorithm, which allows us to compress multidimensional OLAP views derived from big OLAP data cubes that populate Mobile Clouds. In addition to this, IQTS effectively and efficiently supports approximate query answering over compressed multidimensional OLAP views. These functionalities turn to be enabling functionalities for big data applications in a wide collection of modern scenarios, such as healthcare analytics, social network analysis, advanced recommendation systems, and so forth. We provided motivations, anatomy and procedures of IQTS, enriched by several case studies that contributed to pinpoint the benefits coming from our proposed algorithm.

Future work is mostly oriented towards extending our framework with innovative characteristics of big data, such as (i) *advanced machine learning* (e.g., [21,22]); (ii) *semantics-based techniques* (e.g., [23,24]); (iii) *explainability approaches* (e.g., [25,26]); (iv) *privacy-preservation methods over compressed data* (e.g., [27,28,29,30]).

ACKNOWLEDGMENT

This work was funded by the Next Generation EU – Italian NRRP, Mission 4, Component 2, Investment 1.5 (Directorial Decree n. 2021/3277) - project Tech4You n. ECS0000009.

REFERENCES

- [1] Zichuan Xu, Wanli Gong, Qiufen Xia, Weifa Liang, Omer F. Rana, Guowei Wu, “NFV-Enabled IoT Service Provisioning in Mobile Edge Clouds”. *IEEE Trans. Mob. Comput.* 20(5), pp. 1892-1906, 2021
- [2] Hua Deng, Zheng Qin, Qianhong Wu, Zhenyu Guan, Hui Yin, “Revocable Attribute-Based Data Storage in Mobile Clouds”. *IEEE Trans. Serv. Comput.* 15(2), pp. 1130-1142, 2022
- [3] Zichuan Xu, Lizhen Zhou, Sid Chi-Kin Chau, Weifa Liang, Haipeng Dai, Lixing Chen, Wenzheng Xu, Qiufen Xia, Pan Zhou, “Near-Optimal and Collaborative Service Caching in Mobile Edge Clouds”. *IEEE Trans. Mob. Comput.* 22(7), pp. 4070-4085, 2023
- [4] Vijayakumar Varadarajan, Neelanarayanan Venkataraman, Ron Doyle, Imad Fakhri Alshaikhli, Sven Groppe, “Emerging Solutions in Big Data and Cloud Technologies for Mobile Networks”. *Mob. Networks Appl.* 24(3), pp. 1015-1017, 2019
- [5] Mohammad Shorfuazzaman, M. Shamim Hossain, Amril Nazir, Ghulam Muhammad, Atif Alamri, “Harnessing the Power of Big Data Analytics in the Cloud to Support Learning Analytics in Mobile Learning Environment”. *Comput. Hum. Behav.* 92, pp. 578-588, 2019
- [6] Tian Wang, Haoxiong Ke, Xi Zheng, Kun Wang, Arun Kumar Sangaiah, Anfeng Liu, “Big Data Cleaning Based on Mobile Edge Computing in Industrial Sensor-Cloud”. *IEEE Trans. Ind. Informatics* 16(2), pp. 1321-1329, 2020
- [7] Xiaoxu Ren, Chao Qiu, Xiaofei Wang, Zhu Han, Ke Xu, Haipeng Yao, Song Guo, “AI-Bazaar: A Cloud-Edge Computing Power Trading Framework for Ubiquitous AI Services”. *IEEE Trans. Cloud Comput.* 11(3), pp. 2337-2348, 2023
- [8] Dongfen Li, Jinshan Lai, Ruijin Wang, Xiong Li, Pandi Vijayakumar, Brij B. Gupta, Wade Alhalabi, “Ubiquitous Intelligent Federated Learning Privacy-Preserving Scheme under Edge Computing”. *Future Gener. Comput. Syst.* 144, pp. 205-218, 2023
- [9] Hao Jiang, Lixia Li, Haoran Xian, Yulin Hu, Hehe Huang, Juzhen Wang, “Crowd Flow Prediction for Social Internet-of-Things Systems Based on the Mobile Network Big Data”. *IEEE Trans. Comput. Soc. Syst.* 9(1), pp. 267-278, 2022
- [10] Shinwoo Hyun, Seung Mo Yang, Junhwan Kim, Kwang Soo Kim, Jae Hoon Shin, Sang Min Lee, Byun-Woo Lee, Robert M. Beresford, David H. Fleisher, “Development of a Mobile Computing Framework to Aid Decision-Making on Organic Fertilizer Management Using a Crop Growth Model”. *Comput. Electron. Agric.* 181, art. 105936, 2021
- [11] Alfredo Cuzzocrea, “CAMS: OLAPing Multidimensional Data Streams Efficiently”. In: *2009 DaWaK Int. Conf.*, pp. 48-62, 2009
- [12] Alfredo Cuzzocrea, “Compressing Big OLAP Data Cubes in Big Data Analytics Systems: New Paradigms and Future Research Perspectives”. In: *2022 ICSBT Int. Conf.*, p.7, 2022
- [13] Elena N. Stankova, Andrey V. Balakshiy, Dmitry A. Petrov, Vladimir V. Korkhov, Andrey V. Shorov, “OLAP Technology and Machine Learning as the Tools for Validation of the Numerical Models of Convective Clouds”. *Int. J. Bus. Intell. Data Min.* 14(1/2), pp. 254-266, 2019
- [14] Carson K. Leung, Yubo Chen, Calvin S. H. Hoi, Siyuan Shang, Alfredo Cuzzocrea, “Machine Learning and OLAP on Big COVID-19 Data”. In: *IEEE BigData 2020 Int. Conf.*, pp. 5118-5127, 2020
- [15] Vinayakumar Ravi, Tuan D. Pham, Mamoun Alazab, “Attention-Based Multidimensional Deep Learning Approach for Cross-Architecture IoMT Malware Detection and Classification in Healthcare Cyber-Physical Systems”. *IEEE Trans. Comput. Soc. Syst.* 10(4), pp. 1597-1606, 2023
- [16] Alfredo Cuzzocrea, Filippo Furfaro, Domenico Saccà, “Enabling OLAP in Mobile Environments via Intelligent Data Cube Compression Techniques”. *J. Intell. Inf. Syst.* 33(2), pp. 95-143, 2009
- [17] Katrina E. Barkwell, Alfredo Cuzzocrea, Carson K. Leung, Ashley A. Ocran, Jennifer M. Sanderson, James Ayrton Stewart, Bryan H. Wodi, “Big Data Visualisation and Visual Analytics for Music Data Mining”. In: *IEEE IV 2018 Int. Conf.*, pp. 235-240, 2018
- [18] Francesco Buccafurri, Filippo Furfaro, Giuseppe Massimo Mazzeo, Domenico Saccà, “A Quad-Tree Based Multiresolution Approach for Two-Dimensional Summary Data,” *Information Systems* 36(7), pp. 1082-1103, 2011
- [19] G. Colliat, “OLAP, Relational, and Multidimensional Database Systems,” *ACM Sigmod Record* 25(3), pp.64-69, 1996
- [20] Alfredo Cuzzocrea, “Retrieving Accurate Estimates to OLAP Queries over Uncertain and Imprecise Multidimensional Data Streams”. In: *ACM SSDM 2011 Int. Conf.*, pp. 575-576, 2021
- [21] Rocco Langone, Alfredo Cuzzocrea, Nikolaos Skantzos, “Interpretable Anomaly Prediction: Predicting Anomalous Behavior in Industry 4.0 Settings via Regularized Logistic Regression Tools”. *Data Knowl. Eng.* 130, art. 101850, 2020
- [22] Xiaoming Li, Dan Zhang, Ye Zheng, Wuyang Hong, Weixi Wang, Jizhe Xia, Zhihan Lv, “Evolutionary Computation-Based Machine Learning for Smart City High-Dimensional Big Data Analytics”. *Appl. Soft Comput.* 133, art. 109955, 2023
- [23] Alfredo Cuzzocrea, Domenico Saccà, Paolo Serafino, “Semantics-Aware Advanced OLAP Visualization of Multidimensional Data Cubes”. *Int. J. Data Warehous. Min.* 3(4), pp. 1-30, 2007
- [24] Dongxiao He, Yanli Wu, Youyou Wang, Zhizhi Yu, Zhiyong Feng, Xiaobao Wang, Yuxiao Huang, “Identification of Communities With Multi-Semantics via Bayesian Generative Model”. *IEEE Trans. Big Data* 8(4), pp. 869-881, 2022
- [25] Mohanad Sarhan, Siamak Layeghy, Marius Portmann, “Evaluating Standard Feature Sets Towards Increased Generalisability and Explainability of ML-Based Network Intrusion Detection”. *Big Data Res.* 30, art. 100359, 2022
- [26] Gaurav Duggal, Tejas Gaikwad, Bhupendra Sinha, “Dependable Modulation Classifier Explainer with Measurable Explainability”. *Frontiers Big Data* 5, 2022
- [27] Ayman Ibaida, Alsharif Abuadba, Naveen K. Chilamkurti, “Privacy-Preserving Compression Model for Efficient IoMT ECG Sharing”. *Comput. Commun.* 166, pp. 1-8, 2021
- [28] Kamalika Chaudhuri, Chuan Guo, Mike Rabbat, “Privacy-Aware Compression for Federated Data Analysis”. In: *UAI 2022 Int. Conf.*, pp. 296-306, 2022
- [29] Francesco Taurone, Daniel E. Lucani, Qi Zhang, “Zip to Zip-It: Compression to Achieve Local Differential Privacy”. *CoRR abs/2308.14627*, 2023
- [30] Bin Jiang, Jianqiang Li, Huihui Wang, Houbing Song, “Privacy-Preserving Federated Learning for Industrial Edge Computing via Hybrid Differential Privacy and Adaptive Compression”. *IEEE Trans. Ind. Informatics* 19(2), pp. 1136-1144, 2023